

SolarSyncBroker — ghidul repo-ului

Structura proiectului, convenții și cum pornești API-ul.

Backend-ul (REST API) pentru **SolarSyncBroker** — agent autonom de management energetic al unei comunități de 10 case prosumatoare. Lucrare de licență. Spring Boot 3.5 + Java 21 + MySQL.

Plan complet: [PLAN_PROIECT.md](#) (la profesor) · Recapitulare Spring: [RECAP_SPRING.md](#) (la profesor) · Backlog de implementare: [BACKLOG_API.pdf](#) .

Frontend-ul (React + TypeScript, Etapa 9) va fi un repo separat.

Structură

```
solar-sync-broker-api/      acest repo
  src/main/java/ro/mycode/solarsyncbroker/  domeniile (vertical slice)
  src/main/resources/        application.yml + db/migration (Flyway)
  docker-compose.yml         MySQL prin Docker (OPTIONAL, pentru mai târziu)
  pom.xml · mvnw
```

Codul e organizat pe domenii (vertical slice). Fiecare domeniu repetă structura:

```
<domeniu>/
  controller/  endpoint-urile REST
  dtos/        request + response, separate de entități
  exceptions/  excepțiile domeniului
  mapper/      entity <--> dto
  model/       entitățile JPA
  repository/  JpaRepository
  service/
    commandService/  scrieri (create/update/delete)
    queryService/    citiri
  system/          cross-cutting: config, constants, exceptions globale
```

Domeniile (goale, le implementezi pe rând): `auth`, `users`, `house`, `battery`, `simulation`, `market`, `energy`, `grid`, `financial`, `telemetry`, `agent`.

Pornire (Modul 0)

Ai nevoie de **MySQL pornit local** (cum ai la sebi-school) — nu îți trebuie Docker acum.

1. Creează baza de date și userul (o singură dată), în MySQL Workbench sau CLI:

```
CREATE DATABASE solarsync;  
CREATE USER 'solarsync'@'localhost' IDENTIFIED BY 'solarsync';  
GRANT ALL PRIVILEGES ON solarsync.* TO 'solarsync'@'localhost';  
FLUSH PRIVILEGES;
```

2. Pornește API-ul (din rădăcina acestui repo):

```
./mvnw spring-boot:run  
# pornește pe :8080; Flyway rulează (încă fără migrații)
```

Dacă folosești alt user/parolă (ex. `root`), suprascrie prin variabile de mediu — vezi tabelul de la **Configurare**.

Mai târziu (când înveți Docker) poți porni MySQL fără să-l instalezi local: `docker compose up -d mysql` (din acest repo).

Verifici:

```
curl localhost:8080/actuator/health  
# {"status":"UP"}  
  
# Swagger UI (după ce adaugi primul controller, Modul 3):  
# http://localhost:8080/swagger-ui.html
```

Teste

```
./mvnw test
# raport coverage JaCoCo: target/site/jacoco/index.html
```

- Testele **unitare** (formule energetice, SoC, curtailment, validator — partea importantă) sunt JUnit pur, **nu cer Docker, nu cer baza de date**.
- Testele de **integrare** cu Testcontainers pornesc un MySQL real și **cer Docker** — le faci mai târziu (Modul 6), când ajungi la Docker.

Configurare

Parametrii de conexiune se citesc din variabile de mediu, cu valori implicite pentru local:

Variabilă	Implicit
DB_URL	jdbc:mysql://localhost:3306/solarsync
DB_USERNAME	solarsync
DB_PASSWORD	solarsync

Cheile LLM (Modul 8) se citesc **doar** din mediu, niciodată din `application.yml`:

```
AI_PROVIDER=openai | anthropic
OPENAI_API_KEY=...
ANTHROPIC_API_KEY=...
```



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro